



WIF3009

Python for Scientific Computing

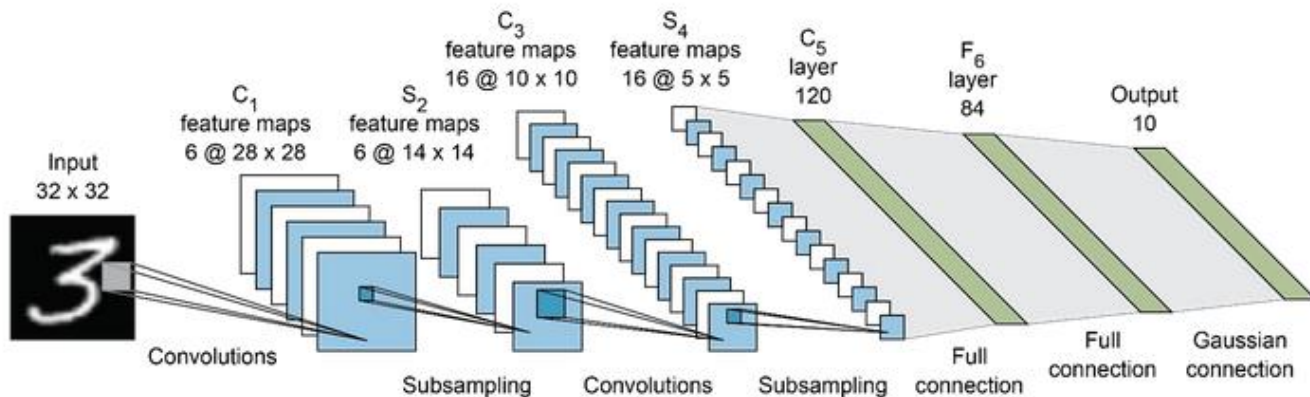
Applied Deep Learning (Part 2)



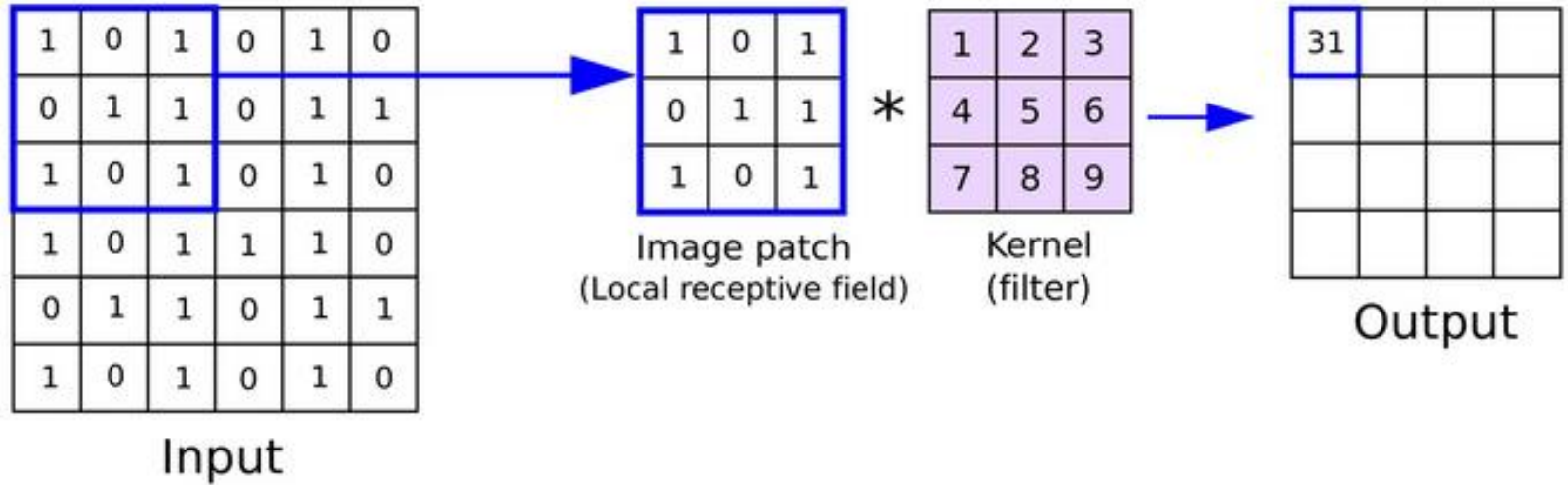
Convolutional Neural Networks (CNN)

CNN (Introduction)

- A Neural Net where the linear operation is a convolution.
- Convolutions are nice because they're invariant to translation
 - Filter doesn't care where in the image, the object of interest is located
 - Forces weights sharing across input pixels, generalizing it even more.



CNN (Introduction)



CNN (Convolutions)

- You have 2 things:
 - Source image
 - CNN kernel filter
- What is a convolution?

Image

I1	I2	I3	I4	I5	I6
I7	I8	I9	I10	I11	I12
I13	I14	I15	I16	I17	I18
I19	I20	I21	I22	I23	I24
I25	I26	I27	I28	I29	I30
I31	I32	I33	I34	I35	I36

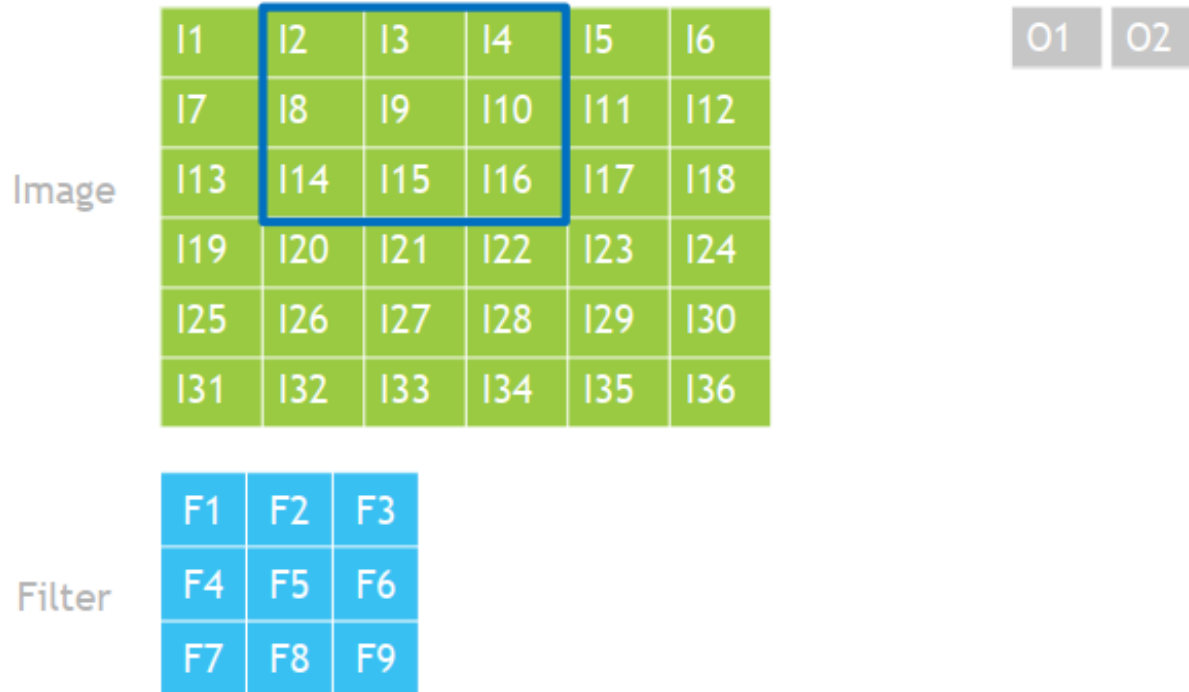
Filter

F1	F2	F3
F4	F5	F6
F7	F8	F9

O1

$$= F1*I1 + F2*I2 + F3*I3 + F4*I7 + F5*I8 + F6*I9 + F7*I13 + F8*I14 + F9*I15$$

CNN (Convolutions)



CNN (Convolutions)

Image

I1	I2	I3	I4	I5	I6
I7	I8	I9	I10	I11	I12
I13	I14	I15	I16	I17	I18
I19	I20	I21	I22	I23	I24
I25	I26	I27	I28	I29	I30
I31	I32	I33	I34	I35	I36

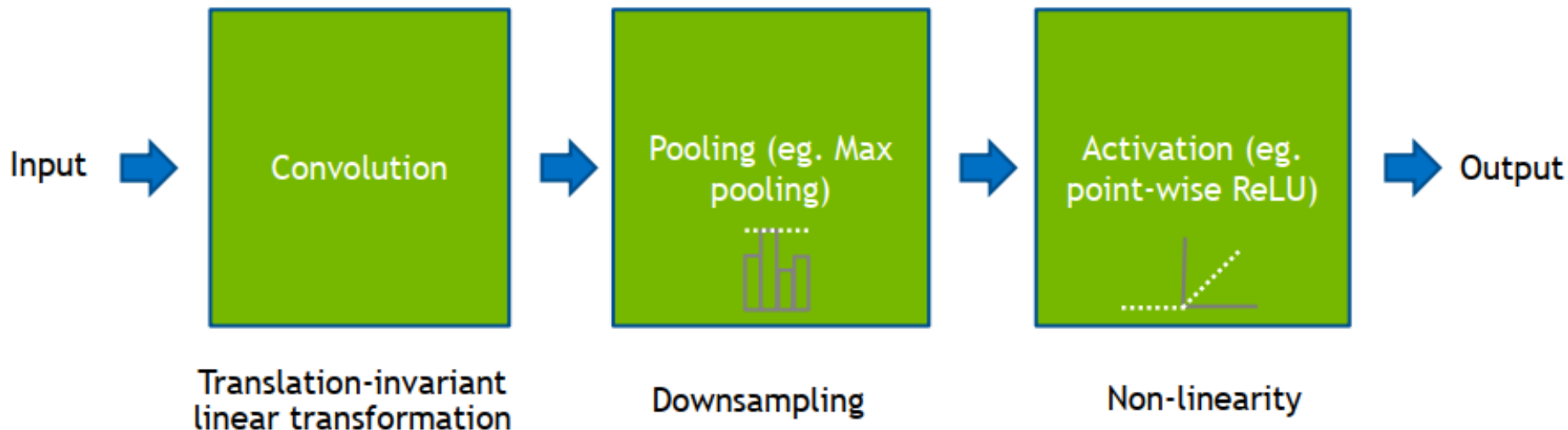
Filter

F1	F2	F3
F4	F5	F6
F7	F8	F9

O1	O2	O3
----	----	----

... and so on until you've covered the entire image

CNN (Three basic components)



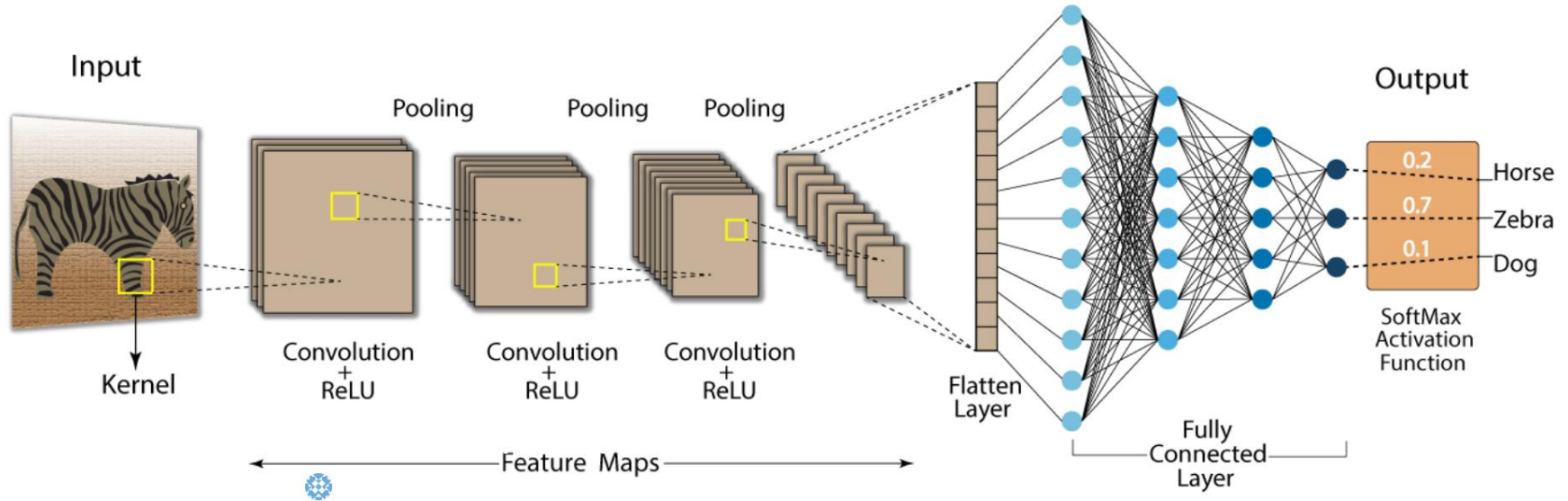
Together, they comprise a non-linear 2-D filter that's at the heart of the CNN

CNN (Three basic components)

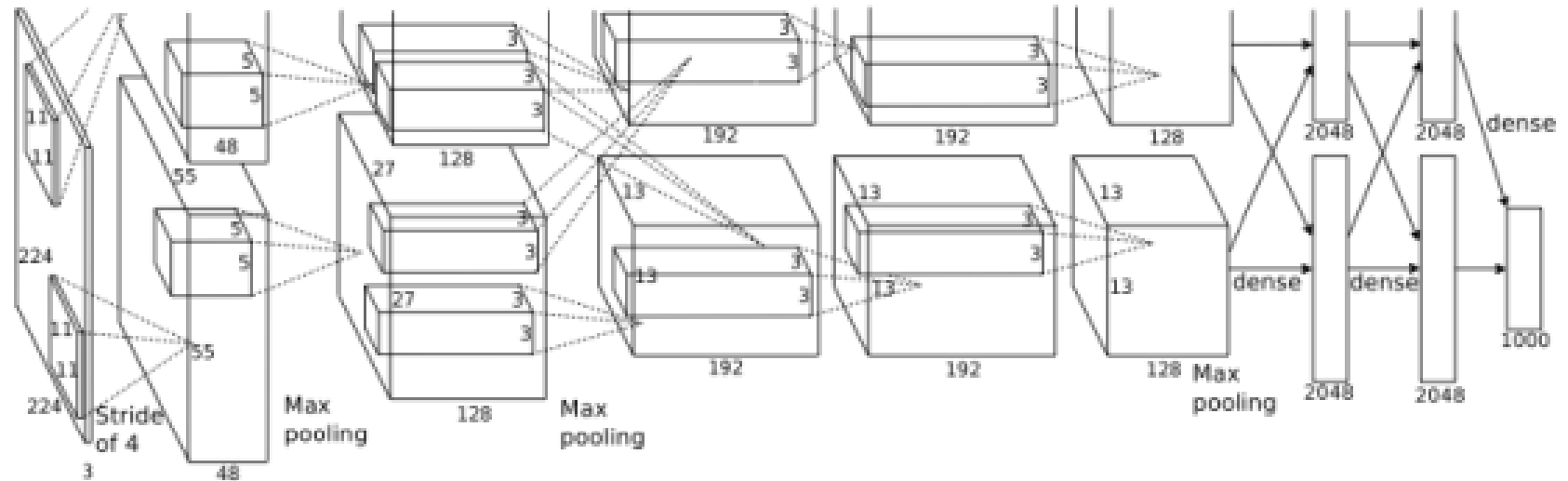


- Natural usages:
 - For image recognition
 - Anything involving pictures or pixels
 - Textual data (YES! You can use CNN with textual data!)

CNN (Anatomy)

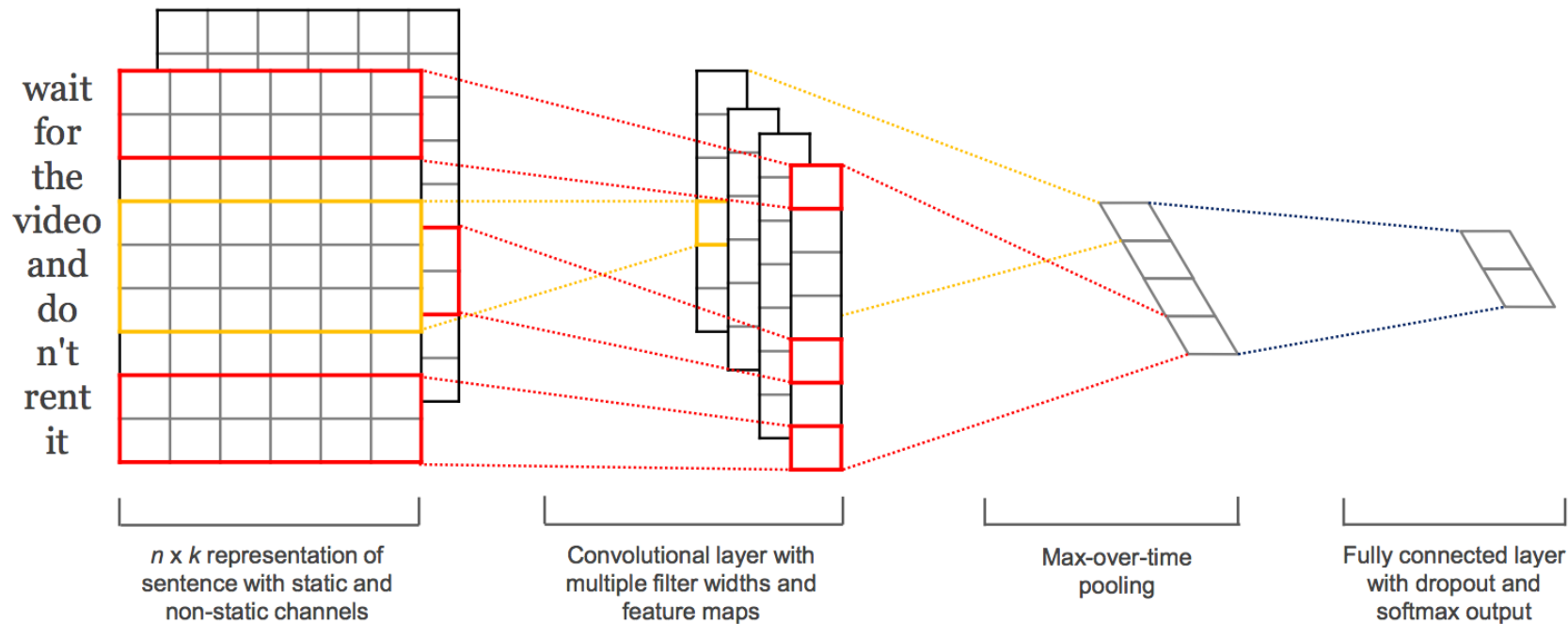


CNN (AlexNet (2012))



A. Krizhevsky, G. Hinton et al. 2012 : Imagenet classification winner

CNN (TextCNN (2014))



CNN



EXERCISE 1 – Convolutional Neural Network

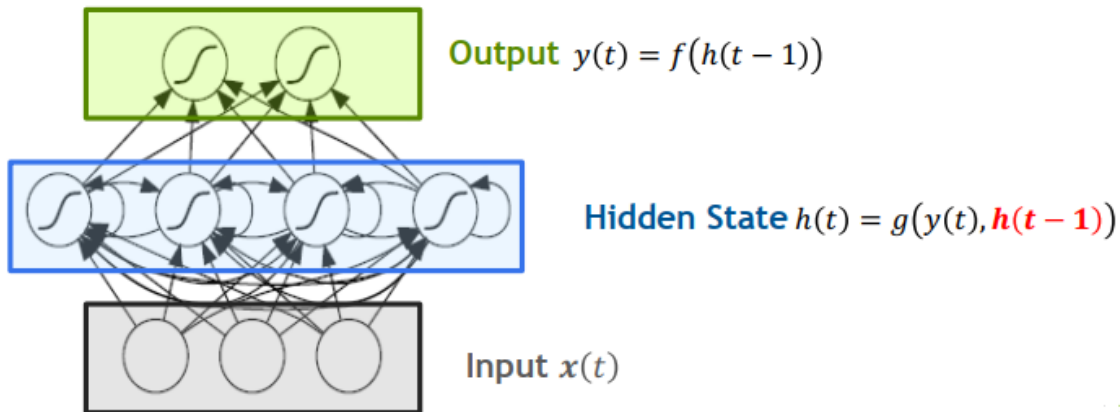
- Packages:
 - Tensorflow
 - Scikit-learn
 - Pillow



Recurrent Neural Networks (RNN)

RNN (Introduction)

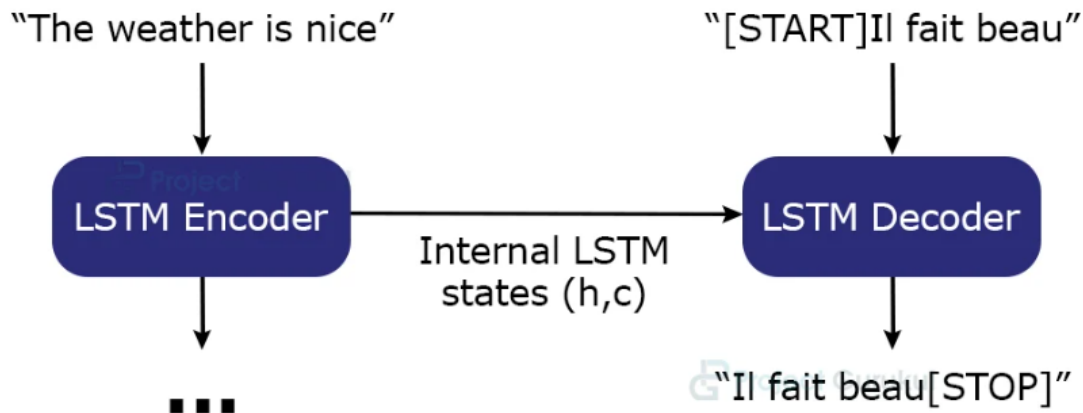
- Why use Recurrent Neural Nets?
 - RNN have hidden states that act as a memory of previous inputs.
- Can predict sequences that has long dependencies.
- For example:
 - Long sentences
 - Stock market charts
 - Signals and patterns



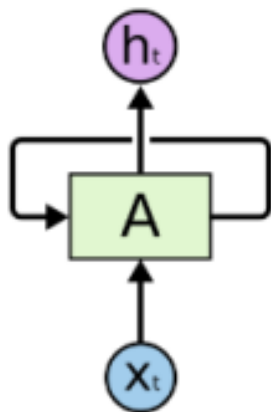
RNN (Introduction)

- Sequence classification and labelling
- Some of them:
 - Handwriting recognition
 - Speech recognition
 - Language translation

my alarm clock did not
my alarm code circle soil rout
alarm code circle soil rout
clock code circle soil rout
did code circle soil rout
not code circle soil rout
must code circle soil rout



RNN (It has memory)



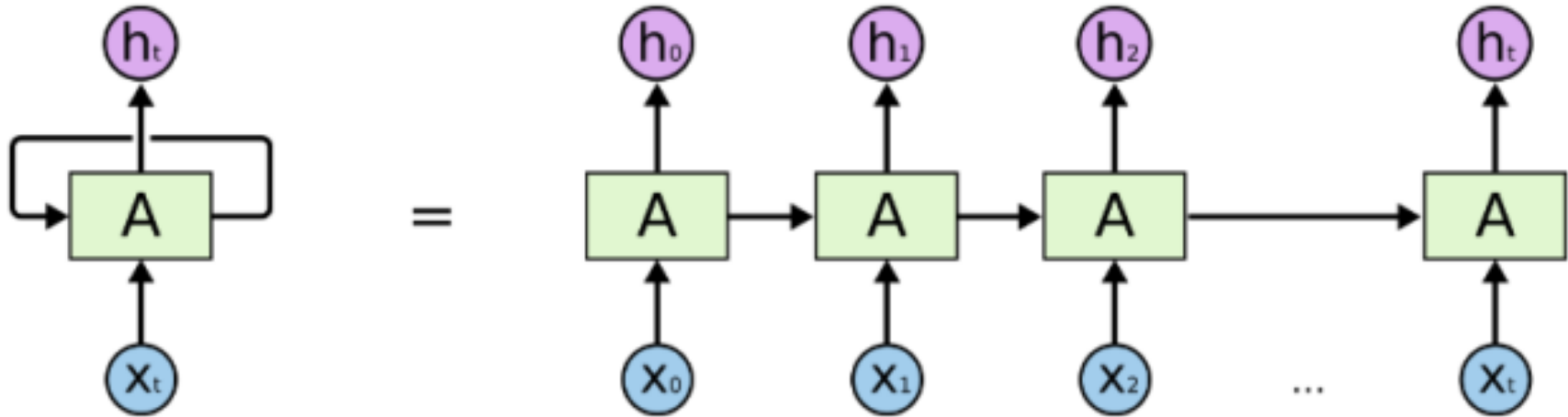
Previous input affects current output

→ We use this for future prediction

$$\underbrace{h_t}_{\text{Current State}} = f_W(\underbrace{h_{t-1}}_{\text{Previous State}}, \underbrace{x_t}_{\text{Current Input}})$$

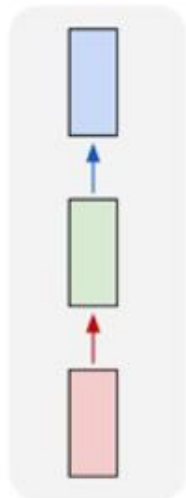
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$y_t = W_{hy}h_t$$

RNN (It has memory)



RNN (Example of flexible RNNs)

one to one



Vanilla
NN

one to many

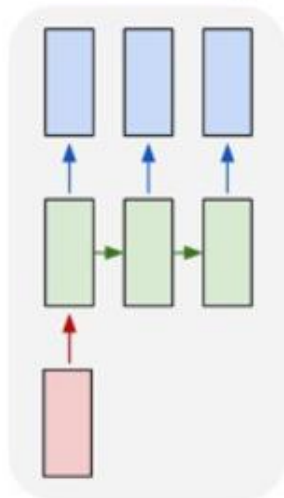
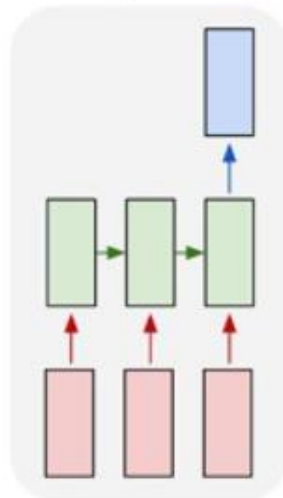


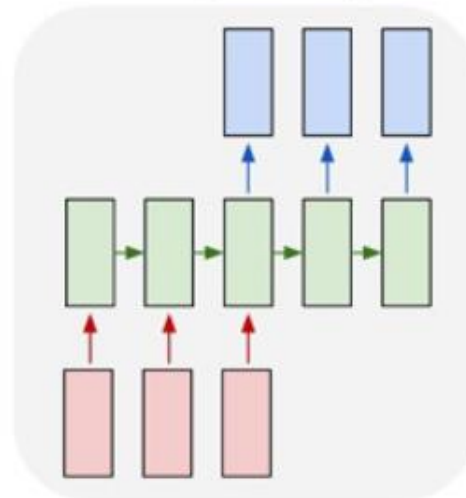
Image
Captioning

many to one



Sentiment
Classification

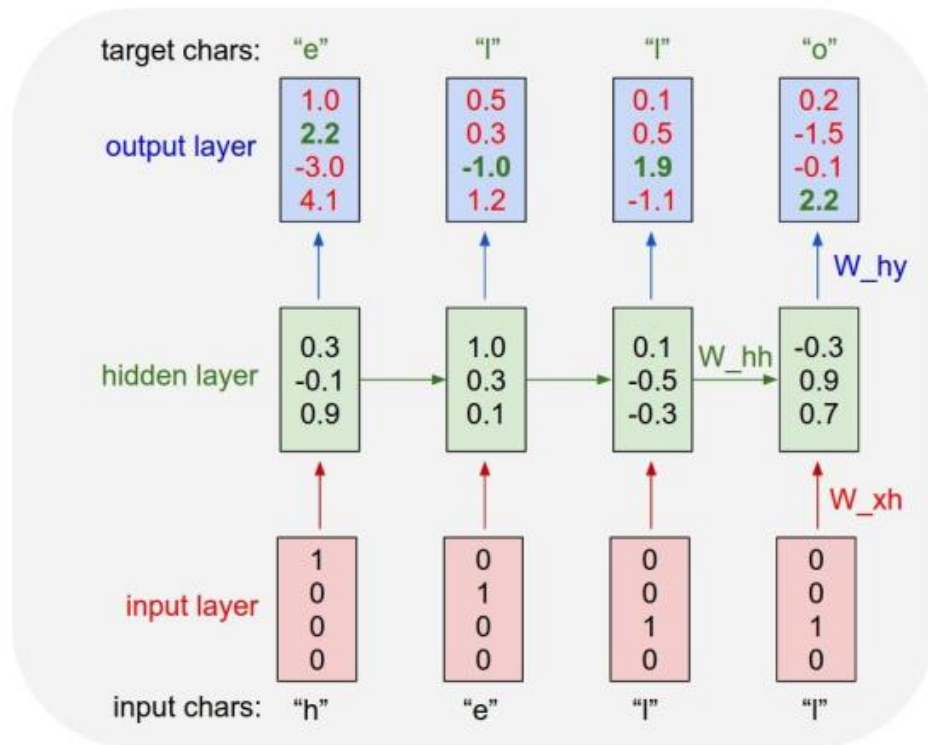
many to many



Machine
Translation

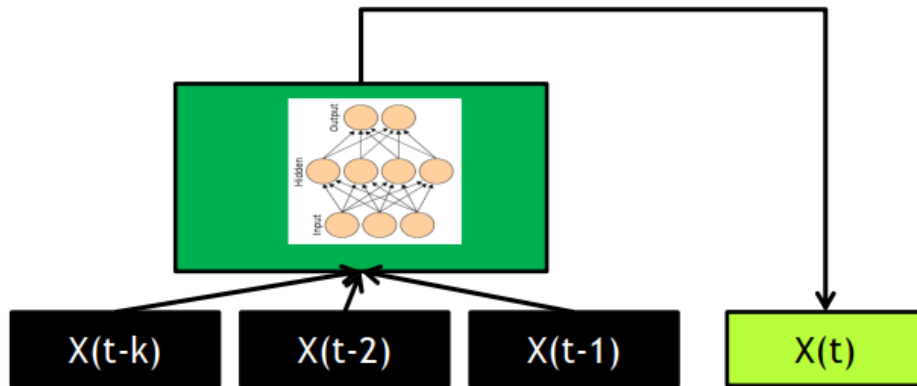
Character-level example

- Vocabulary:
 - [h, e, l, o]
- Example training
Sequence: "Hello"



Feed Forward NN – based Language Model

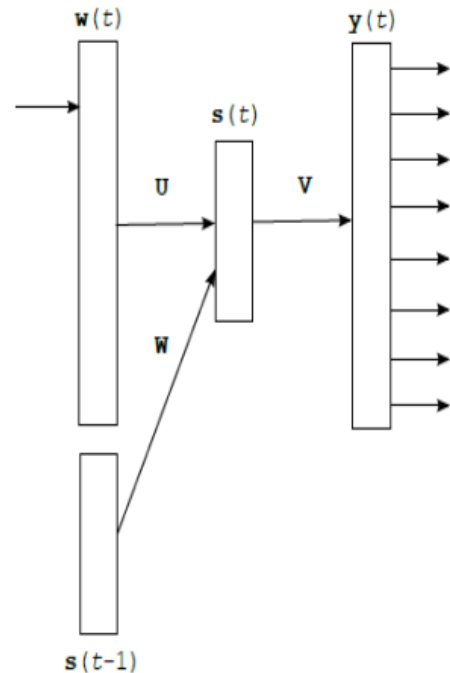
- Feed-forward NN (e.g. Conv. NN) can be used to predict next word from k previous words



- Dictionary = 100,000 $\rightarrow$ NN has 100,000 outputs. Can we scale NN to 100,000 outputs?
- Is there a better way to deal with such a large number of outputs?

RNN – based language model

- RNN allows to extend fixed input window used by TD-NN
- Input layer w and output layer y have the same dimensionality as the vocabulary (10K - 200K).
- Hidden layer s is small (50 - 1000 neurons)
- RNN is trained by unfolding in time and training as a deep feed-forward NN.
- Gradients are propagated back through time



Word Embedding

- The classical NLP regards words as atomic symbols. Each word is represented as one-hot vector [0000010000]
 - motel [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0] AND
 - hotel [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0] = 0
- Word embedding - a word is represented as a dense vector

$$\text{linguistics} = \begin{bmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{bmatrix}$$



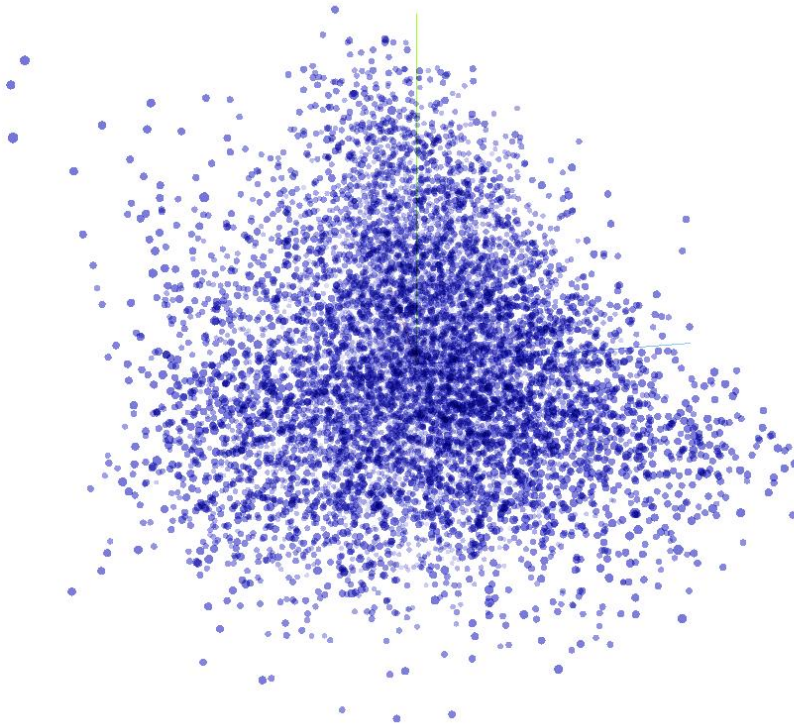
Word Embedding

- Word embedding - a word is represented as a dense vector

$$\text{linguistics} = \begin{bmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{bmatrix}$$



Embedding Projector



<https://projector.tensorflow.org/>

RNN (Example of flexible RNNs)

RNN vs n-gram language models

5-gram: IN TOKYO FOREIGN EXCHANGE TRADING YESTERDAY **THE UNIT** INCREASED AGAINST THE DOLLAR

RNNLM: IN TOKYO FOREIGN EXCHANGE TRADING YESTERDAY **THE YEN** INCREASED AGAINST THE DOLLAR

5-gram: SOME CURRENCY TRADERS SAID THE UPWARD REVALUATION OF THE GERMAN **MARK** WASN'T BIG ENOUGH AND THAT THE MARKET MAY CONTINUE TO RISE

RNNLM: SOME CURRENCY TRADERS SAID THE UPWARD REVALUATION OF THE GERMAN **MARKET** WASN'T BIG ENOUGH AND THAT THE MARKET MAY CONTINUE TO RISE

5-gram: MEANWHILE QUESTIONS REMAIN WITHIN THE E. M. S. **WEATHERED** YESTERDAY'S REALIGNMENT WAS ONLY A TEMPORARY SOLUTION

RNNLM: MEANWHILE QUESTIONS REMAIN WITHIN THE E. M. S. **WHETHER** YESTERDAY'S REALIGNMENT WAS ONLY A TEMPORARY SOLUTION

5-gram: MR. PARNES **FOLEY** ALSO FOR THE FIRST TIME **THE WIND** WITH SUEZ'S PLANS FOR GENERALE DE BELGIQUE'S WAR

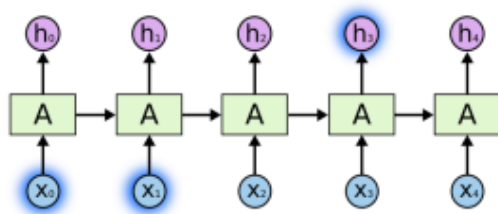
RNNLM: MR. PARNES **SO LATE** ALSO FOR THE FIRST TIME **ALIGNED** WITH SUEZ'S PLANS FOR GENERALE DE BELGIQUE'S WAR

5-gram: HE SAID THE GROUP WAS **MARKET** IN ITS STRUCTURE AND NO ONE HAD LEADERSHIP

RNNLM: HE SAID THE GROUP WAS **ARCANE** IN ITS STRUCTURE AND NO ONE HAD LEADERSHIP

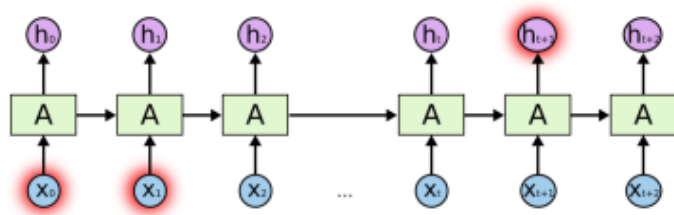
RNN (Problem with RNNs)

- Problem of long-term dependencies!
 - Vanishing gradient problem
 - Gradient exploding problem



RNN has *short memory*: the influence of a given input on the hidden layer, and therefore on the network output, either decays or blows up exponentially:

$$\begin{aligned}h(1) &= W_1 * \mathbf{x(1)} + W_2 * h(0) \\h(2) &= W_1 * x(2) + W_2 * h(1) \\&= W_1 * x(2) + W_2 * (W_1 * x(1) + W_2 * h(0)) \\&= W_1 * x(2) + \mathbf{W_2 * W_1 * x(1)} + W_2^2 * h(0) \\h(k) &= W_1 * x(k) + W_2 * W_1 * x(k-1) + \dots \\&\quad + \mathbf{W_2^{k-1} * W_1 * x(1)} + W_2^k * h(0)\end{aligned}$$



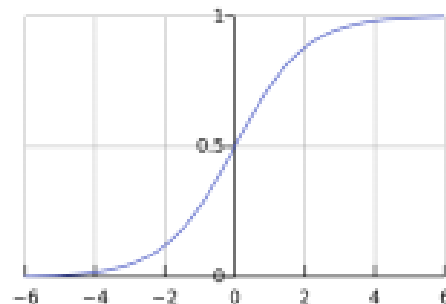
RNN (Long Short-term Memory (LSTM))

- Enter: Long Short-Term Memory (LSTM)
- Introduced to address RNN short memory.
- Has a new basic element called **memory cell**.

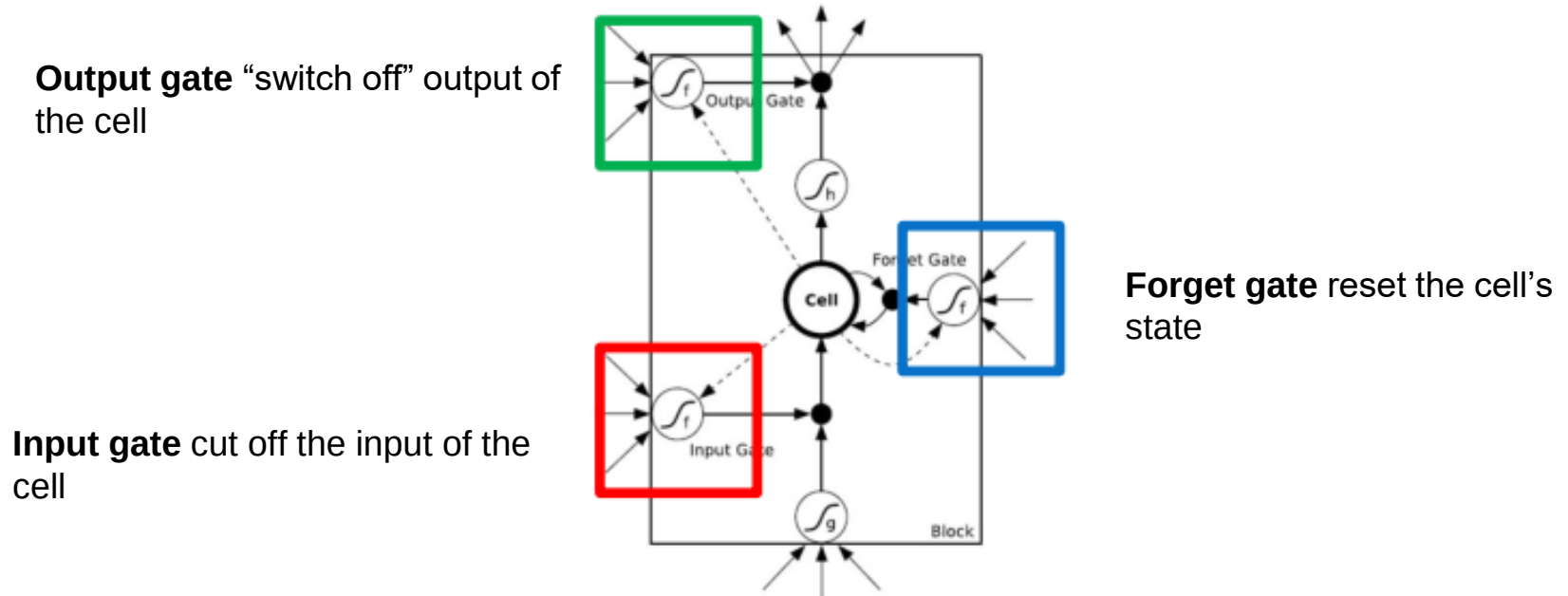
All gates use logistic sigmoid:

0 - gate closed,

1 - gate open

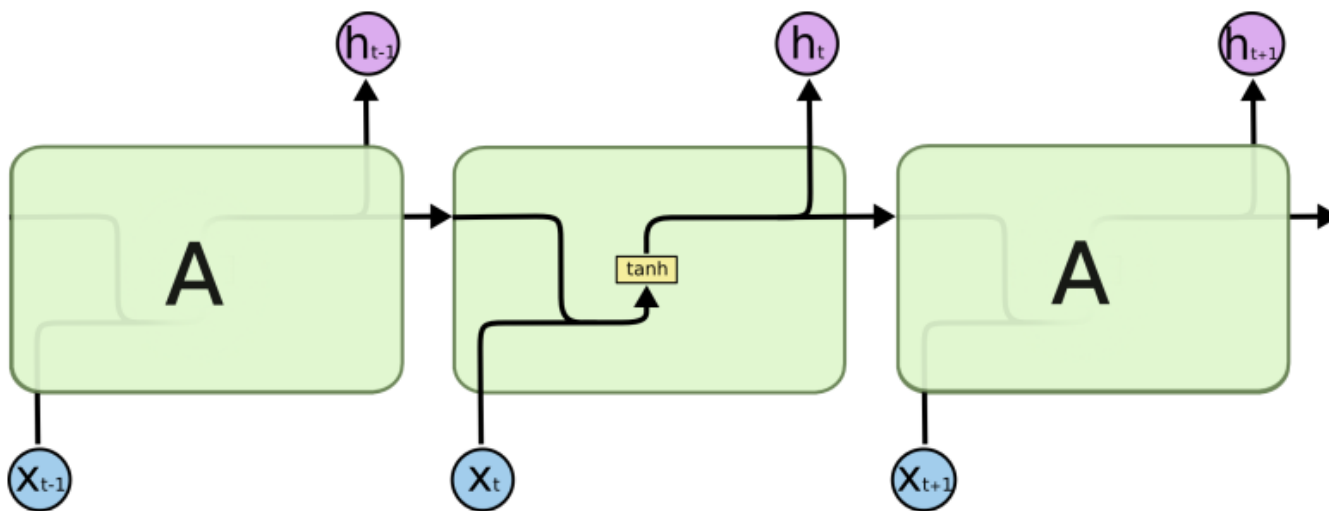


RNN (Long Short-term Memory (LSTM))



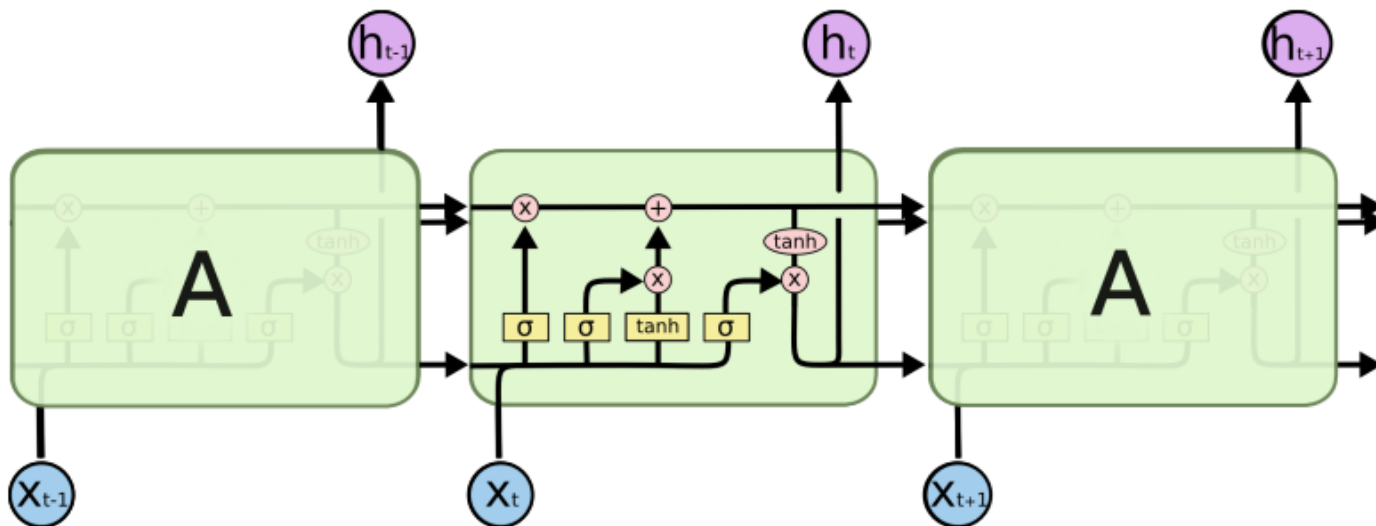
RNN (Long Short-term Memory (LSTM))

RNN Sequence Representation



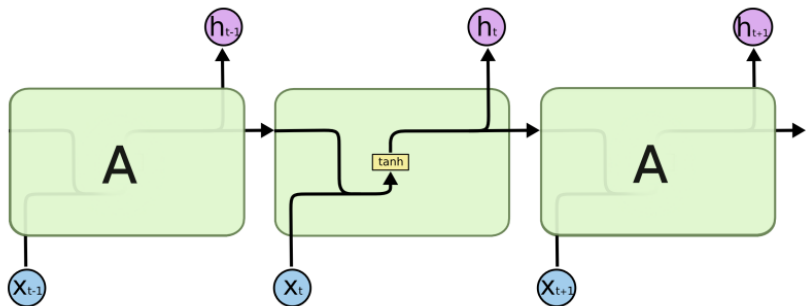
RNN (Long Short-term Memory (LSTM))

LSTM representation

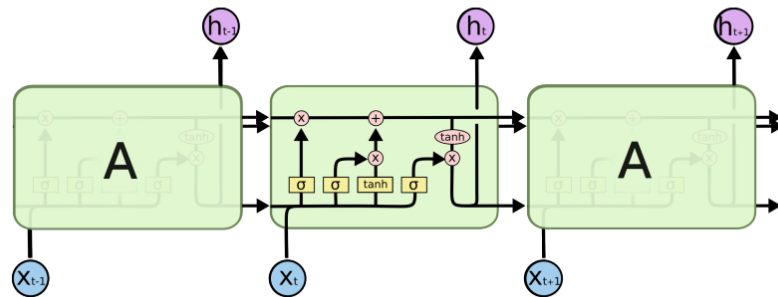


RNN (Long Short-term Memory (LSTM))

RNN Sequence Representation



LSTM representation





Beyond this..... what?

What is wrong with CNN/RNN ?



An RNN has two major disadvantages

- It's slow to train.
- Long sequences lead to vanishing gradient or the problem of long-term dependencies. In simple terms, its memory is not that strong when it comes to remembering old connections.

What about CNN:

- Vanishing gradient
- Slow training



Now enter...

Transformers

- Transformers are a type of neural network architecture, particularly adept at processing sequential data like text, audio, and video.
- They leverage a mechanism called **attention**, which allows them to focus on relevant parts of the input when making predictions or generating new data.
- Unlike older architectures like RNNs and CNNs, transformers **can process entire sequences in parallel**, making them efficient and capable of capturing long-range dependencies.



Transformers



I'm built different

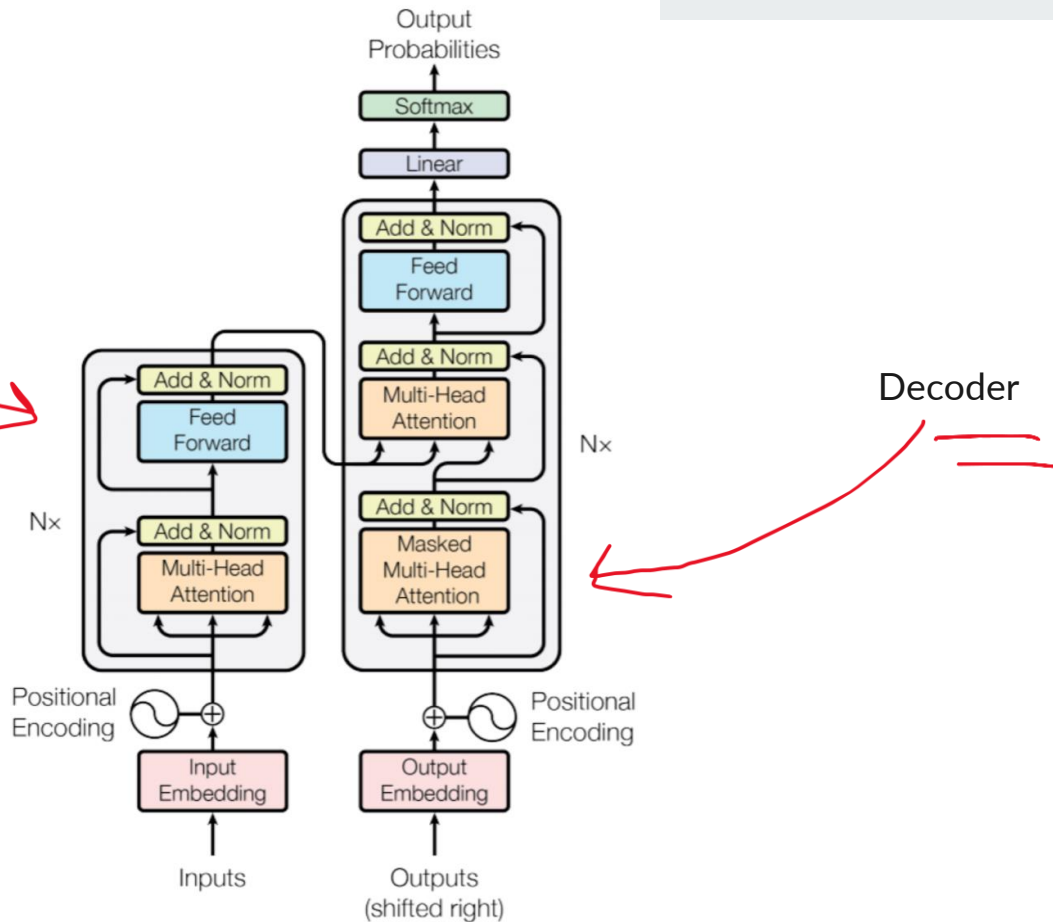


Transformers

Encoder

The Transformer
architecture

By Google Brain

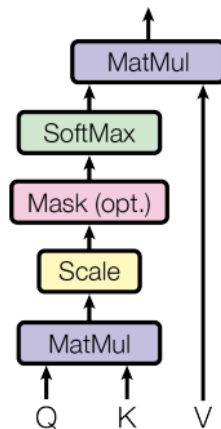


Transformers – Attention Mechanism

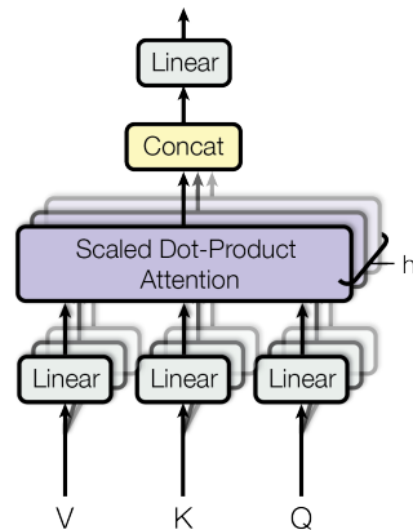
Imagine you're trying to find the most relevant information from a bunch of books.

- Query:** This is like your question.
- Keys:** Each book has a keyword or a summary (its "key") that tells you what it's about.
- Values:** The actual content of each book is its "value."

Scaled Dot-Product Attention



Multi-Head Attention



Transformers – Attention Mechanism



The attention mechanism works like this:

1. It compares your **query** (your question) to the **key** of each book.
2. Based on how well your question matches each book's key, it gives each book a "score" or "weight." A higher score means a better match.
3. Finally, it takes a little bit of the content (the **value**) from each book, but more from the books with higher scores, and combines them to give you the most relevant answer (the **output**).

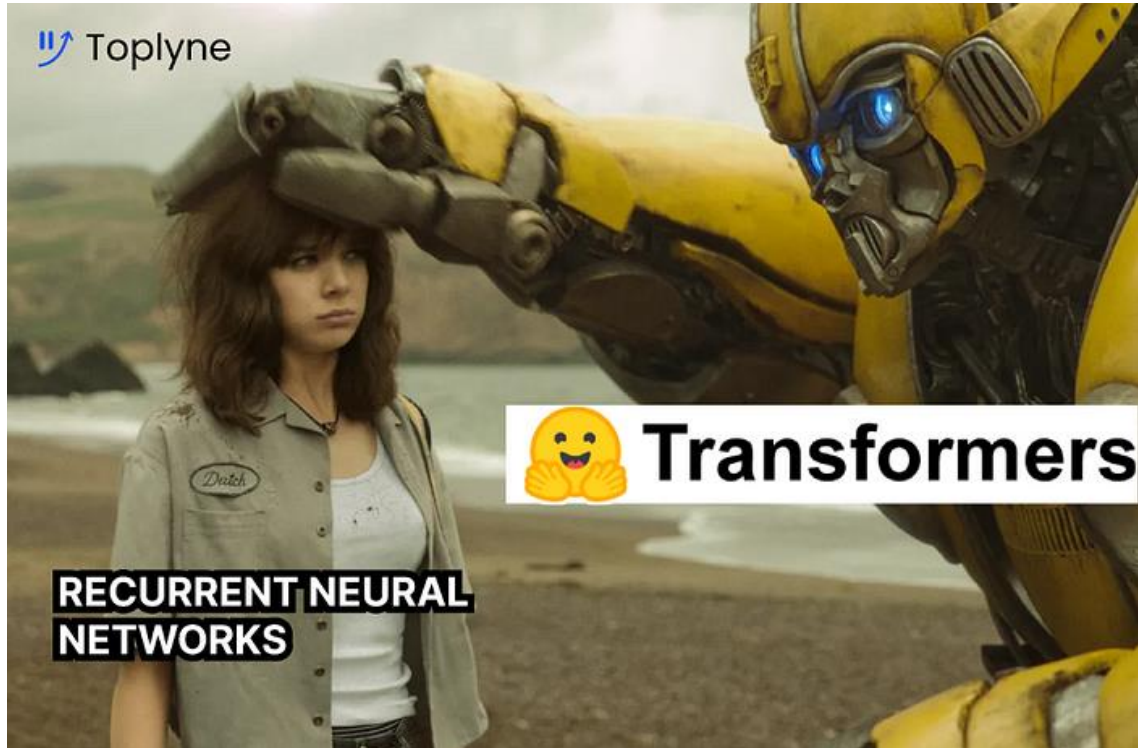
Transformers – Attention Mechanism



The attention mechanism works like this:

- 1.It compares your **query** (your question) to the **key** of each book.
- 2.Based on how well your question matches each book's key, it gives each book a "score" or "weight." A higher score means a better match.
- 3.Finally, it takes a little bit of the content (the **value**) from each book, but more from the books with higher scores, and combines them to give you the most relevant answer (the **output**).

Transformers – There..there RNNs





**But what can you do with
Transformers?**



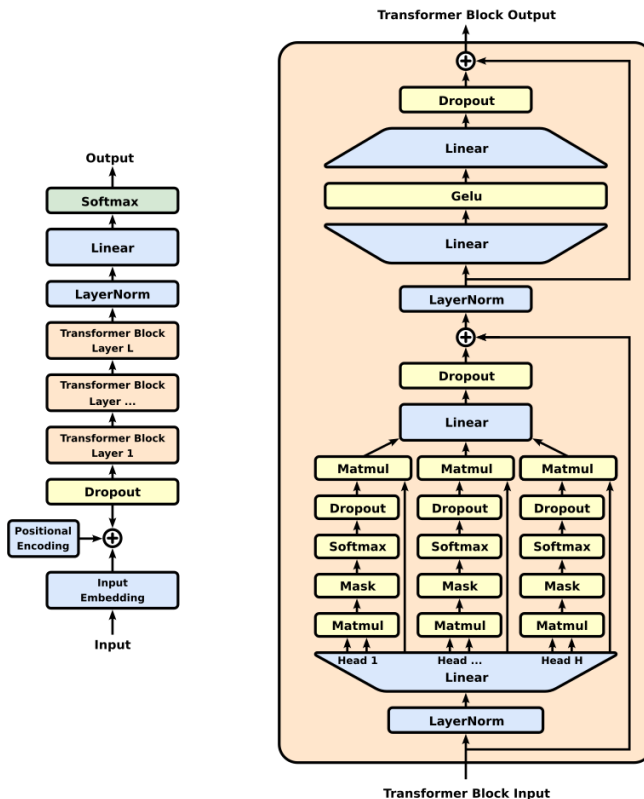
Well.....Large Language Models



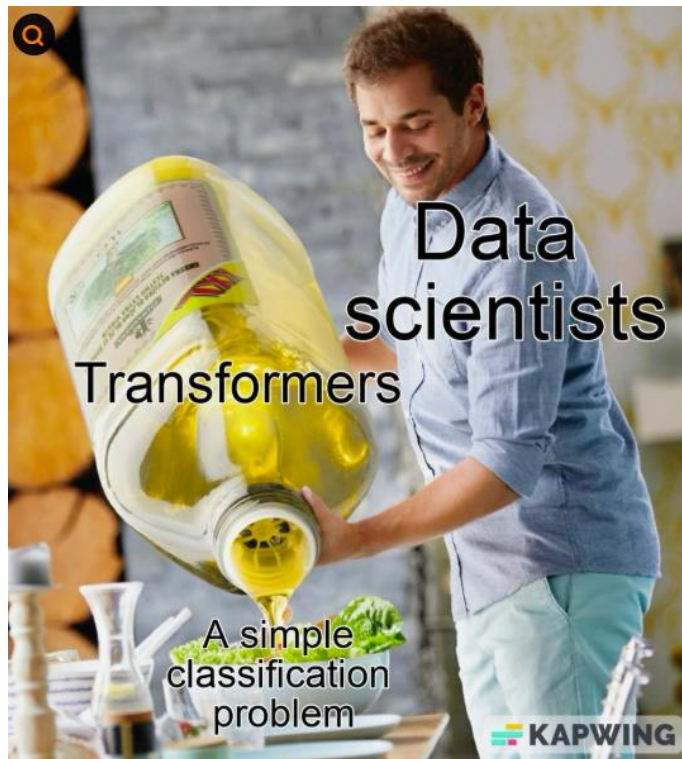
Transformers – Generative Pre-trained Transformer

Generative Pre-trained
Transformer (GPT)

By OpenAI



Transformers – LLMs





Tips and Tricks in ML/DL

Remember hyper-parameter tuning?



- Hyper-parameters are parameters that are not directly learnt within estimators.
- In scikit-learn they are passed as arguments to the constructor of the estimator classes.
- Typical examples include C, kernel and gamma for Support Vector Classifier, alpha for Lasso, etc.

```
estimator.get_params()
```

- It is possible and recommended to search the hyper-parameter space for the best cross validation score.

Do it automatically using GridSearchCV



- The grid search provided by GridSearchCV exhaustively generates candidates from a grid of parameter values specified with the `param_grid` parameter. For instance, the following `param_grid`:

```
sklearn.svm.SVC());
```

```
param_grid = [  
    {'C': [1, 10, 100, 1000], 'kernel': ['linear']},  
    {'C': [1, 10, 100, 1000], 'gamma': [0.001, 0.0001], 'kernel': ['rbf']},  
]
```

specifies that two grids should be explored

What if your dataset is small ?



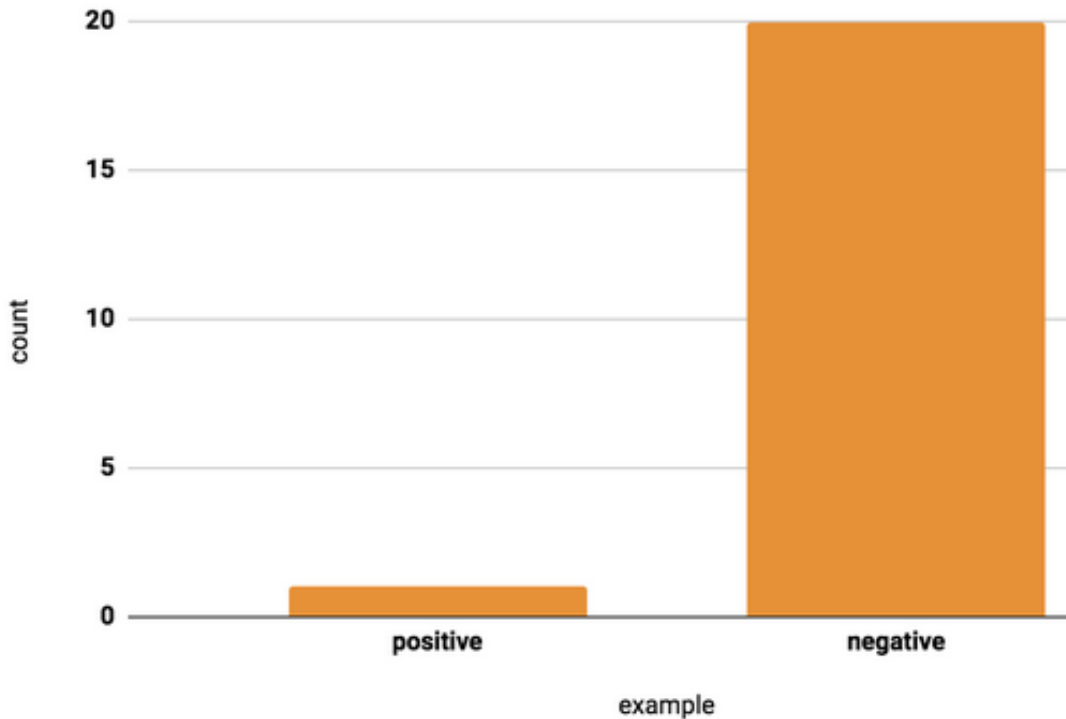
- If your data has less than 500 samples or datapoints.
- Worth it to split 80/10/10 into training/validation/testing sets?
- Nope.

- What should you do?
- Just use CROSS VALIDATION (CV)

What if your data is imbalanced ?



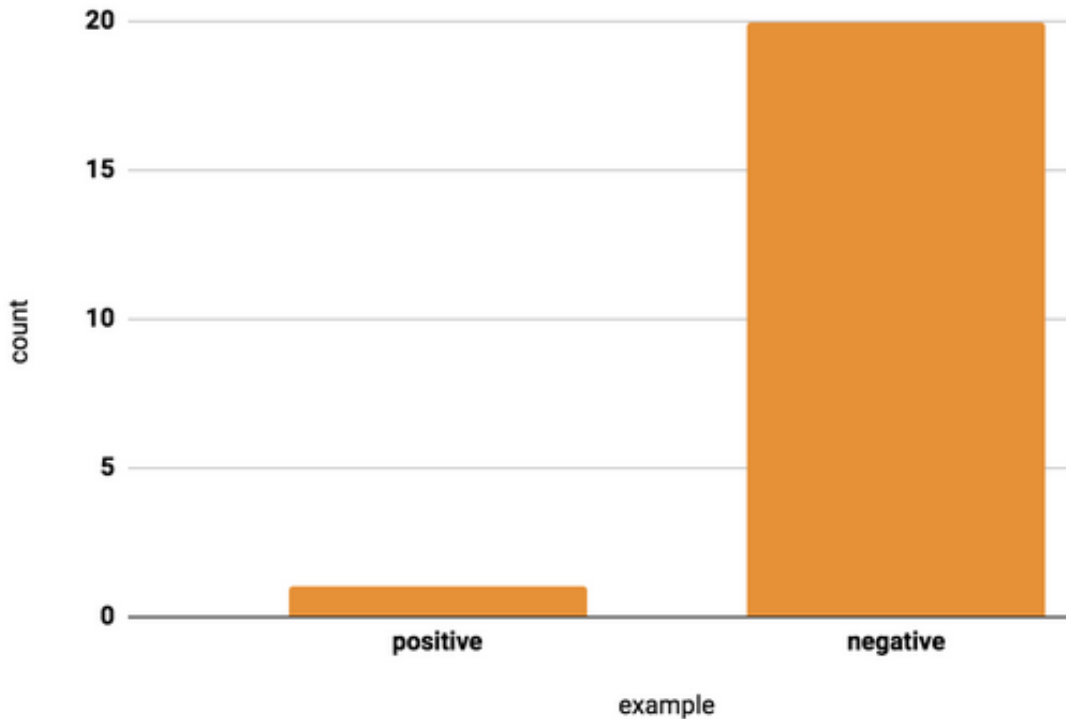
- What is imbalanced data?
- The distribution of classes/categories is uneven or too one-sided.



What if your data is imbalanced ?

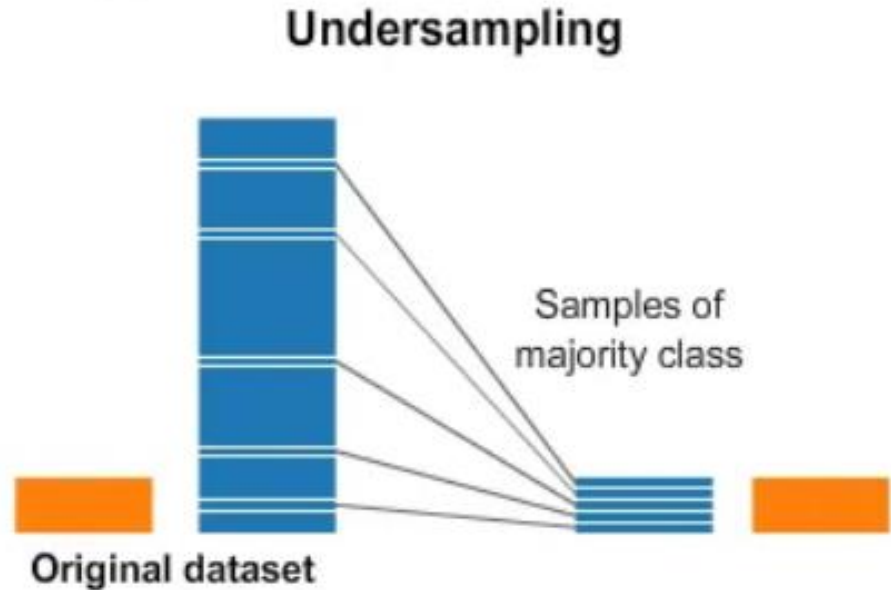


- What to do?
- Try either this:
 - Undersampling
 - Oversampling
- Undersampling is easier!



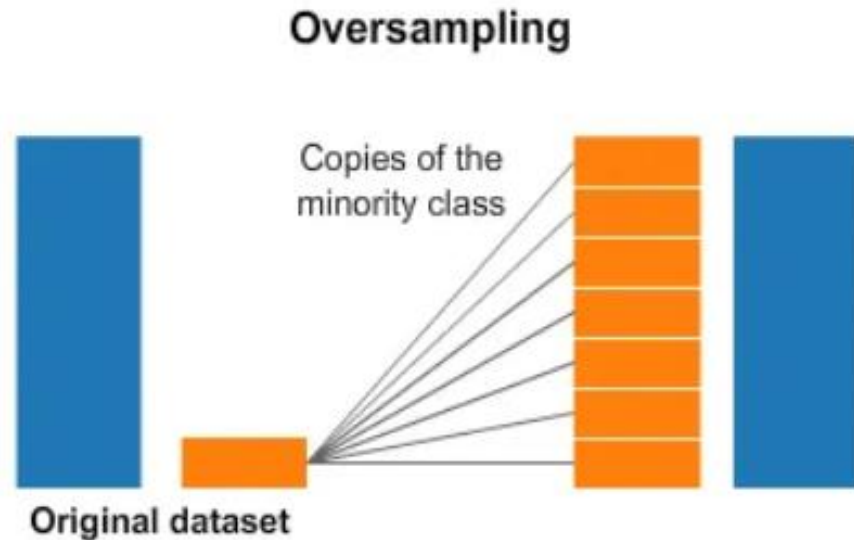
What if your data is imbalanced ? - Undersampling

- Get all the data from minority class
- Samples the same/similar amount of data from the majority class.
- Done.
- Now your new data is “balanced” .



What if your data is imbalanced ? - Oversampling

- Get all/most of the data from the majority class.
- Now, create fake/synthetic data from the minority class.
- How? Use technique like:
 - SMOTE
https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html
- Fake data created and mixed together.
- So now it is a “balanced” data.



What if your data is imbalanced ? – Hmmm.....

- But the tricky thing is life is always imbalanced and unfair.
- Therefore, the usual distribution of data that you will see will most probably be imbalanced.
- Example:
 - Normal traffic > accidents
 - Good people > bad people
 - Normal app > malware

